

Technical University of Lodz

Currency Analysis System

Software Design Document

Jerzy Andrusiewicz 251118, Karolina Banaszewska
251123, Wojciech Korbel 251169, Jędrzej Niedzielski
251193, Nina Zugaj 251254
2026-05-25

Table of Contents

Acceptance Test Report	2
List of Reported Bugs.....	2
System Architecture Description.....	2
Testing and CI Layer.....	3
Repository and Version Control	3
Component Diagram	3
Sequence Diagram	4
Activity Diagram	5
List of Figures	7

Acceptance Test Report

The purpose of the tests was to verify the compliance of the implementation with the functional requirements and to identify inconsistencies affecting the correctness of the system's operation, particularly in the areas of statistics generation, input data validation, and results presentation.

The tests were performed manually and involved the execution of end-user scenarios.

All identified issues were reported in GitHub as Bug / Issue entries in the project repository.

List of Reported Bugs

During the acceptance testing phase, the following issues were recorded:

- Table headers differ from specification (Issue #55)
- Incorrect unit formatting in text-based statistical messages (Issue #54)
- Histogram title differs from specification (Issue #53)
- Histogram generation should not be allowed for identical currency pairs (Issue #52)
- Missing validation for future date selection (Issue #51)
- Incorrect time range calculations for statistical analysis (Issue #50)
- List of currencies differs from specification (Issue #48)

Each issue included a title clearly defining the described defect. Some issues were additionally supplemented with comments containing relevant excerpts from the system specification, allowing for direct reference to requirements and verification of implementation compliance.

System Architecture Description

The Currency Analysis System (CAS) has been designed as a Single Page Application (SPA).

The frontend has been implemented using:

- React
- TypeScript
- Material Design

The user interface runs in a web browser and communicates with an external API to retrieve currency data.

The system communicates directly with the NBP Web API, from which both current and historical exchange rate data are retrieved. Communication is performed using HTTP requests executed on the frontend side.

Testing and CI Layer

The system uses automated unit tests executed within a Continuous Integration (CI) process.

Technologies used include Vitest and GitHub Actions. The CI process automatically installs project dependencies, runs automated tests, and verifies the correctness of the application before allowing changes to be merged into the main, develop, and release branches.

Repository and Version Control

The project source code is stored in a GitHub repository and managed using the Git version control system. Changes are introduced via Pull Requests and a Code Review process.

Component Diagram

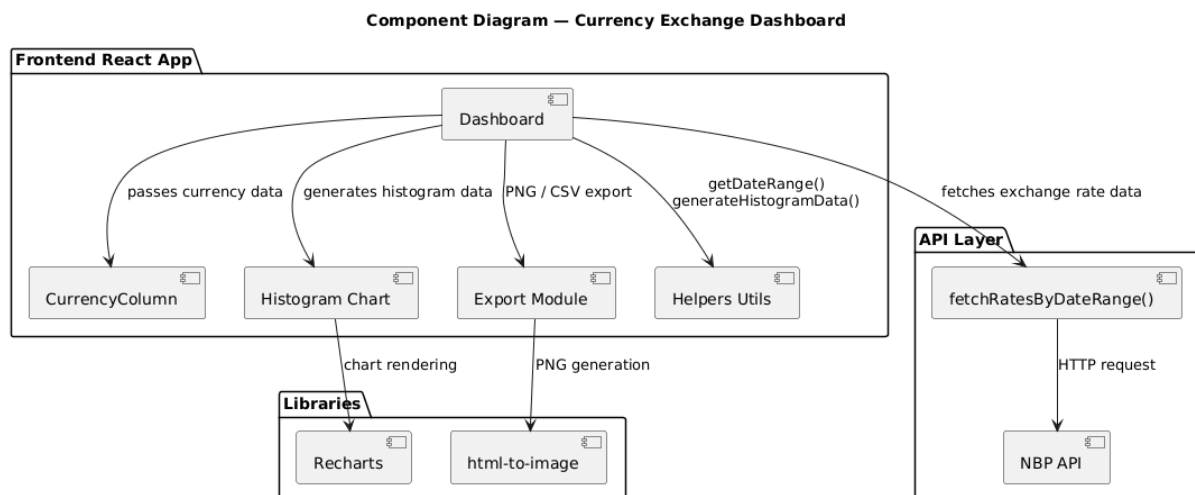


Figure 1. Component Diagram of the Currency Exchange Dashboard.

The diagram above illustrates the high-level structural architecture of the application. The system is divided into a main Frontend React App containing the core user interface and logic including the Dashboard, visual components, and export modules. It interacts with an API Layer, which handles HTTP requests to fetch external exchange rate data via the NBP API. Additionally, the system utilizes external Libraries like Recharts for rendering charts and html-to-image for PNG generation to support data visualization and file export features.

Sequence Diagram

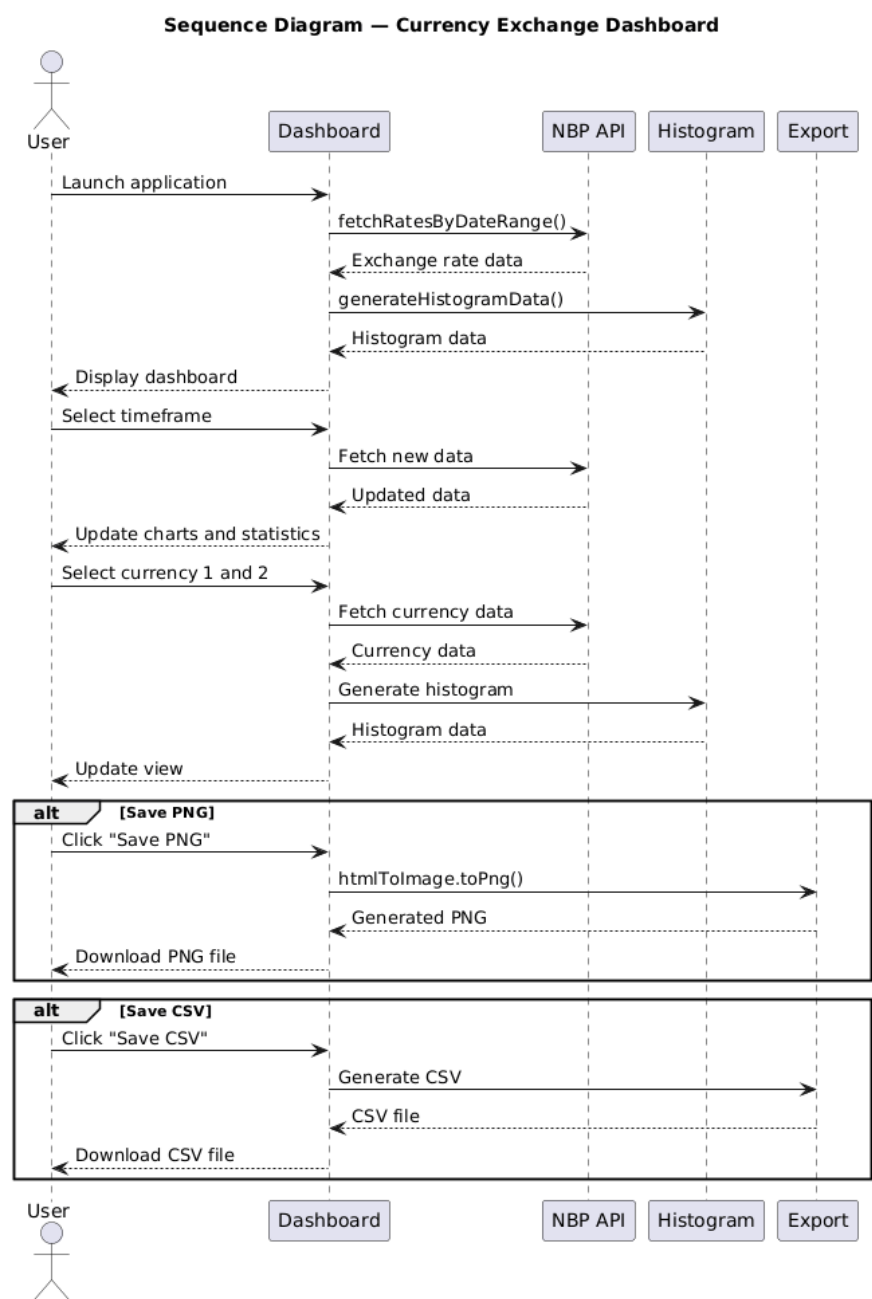


Figure 2. Sequence Diagram illustrating user interactions and system data flow.

This sequence diagram depicts the chronological message flow between the user, the dashboard interface, and the underlying services. It details the step-by-step execution of core functionalities, starting from the initial launch and automated data fetching from the NBP API, through dynamic histogram generation based on user input. Furthermore, it explicitly models the alternative operational pathways for the export module, showing the exact execution flow when a user decides to save a chart as a PNG or export raw data to a CSV file.

Activity Diagram

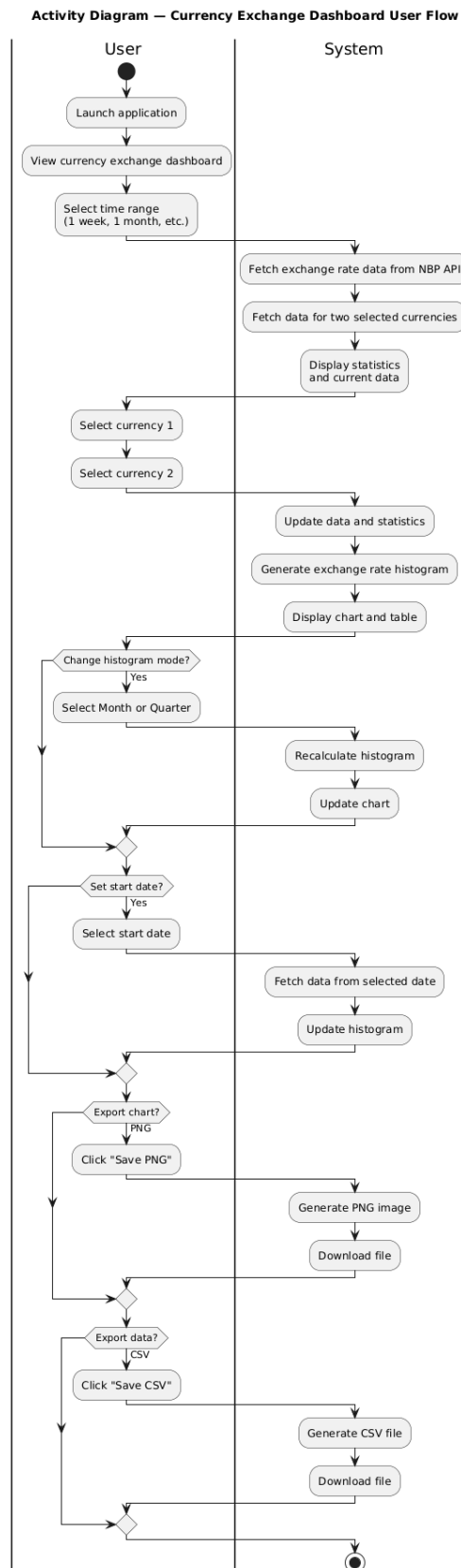


Figure 3. Activity Diagram modeling the dashboard's user flow and system responses.

The activity diagram utilizes swimlanes to map the dynamic behavior of the system, clearly separating user actions from automated system processes. It visualizes the complete operational workflow from launching the application and selecting specific time ranges or currencies, to the system's corresponding background actions. These automated responses include fetching specific datasets, recalculating statistics, updating the histogram view, and sequentially handling file generation and downloads during the export process.

List of Figures

Figure 1. Component Diagram of the Currency Exchange Dashboard.	2
Figure 2. Sequence Diagram illustrating user interactions and system data flow.	3
Figure 3. Activity Diagram modeling the dashboard's user flow and system responses. .	4